



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 3

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «**Методы многомерного поиска**»

Студент	<u>ИУ5И-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Методы оптимизации играют важную роль в задачах машинного обучения, так как многие алгоритмы обучения сводятся к минимизации некоторой целевой функции или функции потерь. В данной лабораторной работе рассматривается задача безусловной многомерной оптимизации, в которой необходимо найти минимум тестовой функции Розенброка.

Цель лабораторной работы — изучение и программная реализация алгоритмов многомерного поиска первого и второго порядка, а также сравнение их эффективности по количеству итераций, времени выполнения и точности найденного решения.

В ходе работы были реализованы методы сопряженных градиентов Флетчера–Ривза и Полака–Рибьера, а также квазиньютоновские методы DFP, BFGS и L-BFGS. Для каждого метода был выполнен поиск минимума функции, рассчитаны значения целевой функции, нормы градиента и построены графики сходимости.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо найти минимум тестовой функции Розенброка методами многомерной оптимизации. В работе рассматривается вариант 2, для которого заданы следующие параметры: $a = 150$, $b = 2$, $f_0 = 100$, $n = 3$.

Тестовая функция имеет вид:

$$f(x) = \sum_{i=1}^{n-1} a(x_{i+1} - x_i^2)^2 + b(x_i - 1)^2 + f_0.$$

Для выбранного варианта при $n = 3$ функция принимает вид:

$$f(x_1, x_2, x_3) = 150(x_2 - x_1^2)^2 + 2(x_1 - 1)^2 + 150(x_3 - x_2^2)^2 + 2(x_2 - 1)^2 + 100.$$

В ходе выполнения лабораторной работы необходимо реализовать следующие методы:

- метод Флетчера–Ривза;
- метод Полака–Рибьера;

- метод Дэвидона–Флетчера–Пауэлла (DFP);
- метод BFGS;
- метод L-BFGS.

Также необходимо найти стационарную точку функции, вычислить значение функции в этой точке, оценить скорость сходимости алгоритмов и сравнить методы по количеству итераций и времени выполнения.

3) Текст программы и экранные формы с примерами выполнения

3.1. Импорт необходимых библиотек

На первом этапе были подключены основные библиотеки, необходимые для выполнения лабораторной работы. Для численных вычислений и работы с векторами использовалась библиотека `numpy`. Для формирования сравнительной таблицы результатов была подключена библиотека `pandas`. Для построения графиков сходимости использовалась библиотека `matplotlib`. Также была подключена функция `minimize_scalar` из библиотеки `scipy.optimize`, которая применялась для выполнения одномерного поиска шага вдоль направления спуска.



Импорт необходимых библиотек для выполнения лабораторной работы

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from scipy.optimize import minimize_scalar

print("Libraries imported successfully")
```

... Libraries imported successfully

Рисунок 1 — Импорт необходимых библиотек

3.2. Задание параметров варианта и определение функции Розенброка

На следующем этапе были заданы параметры варианта 2: $a = 150$, $b = 2$, $f_0 = 100$, $n = 3$. После этого была определена тестовая функция Розенброка, используемая для проверки работы алгоритмов многомерной оптимизации.

Так как $n = 3$, функция зависит от трех переменных x_1 , x_2 и x_3 . Теоретически минимум данной функции достигается в точке $x^* = (1, 1, 1)$. При подстановке этой точки значение функции равно $f(x^*) = 100$. Это значение далее используется для проверки корректности работы реализованных алгоритмов.

```
▶ # Задание параметров варианта и определение тестовой функции Розенброка

a = 150
b = 2
f0 = 100
n = 3

def rosenbrock_function(x):
    x = np.array(x, dtype=float)
    value = 0
    for i in range(n - 1):
        value += a * (x[i + 1] - x[i] ** 2) ** 2 + b * (x[i] - 1) ** 2
    return value + f0

x_star_theoretical = np.ones(n)
f_star_theoretical = rosenbrock_function(x_star_theoretical)

print("Variant:", 2)
print("a =", a)
print("b =", b)
print("f0 =", f0)
print("n =", n)
print("Theoretical minimum point:", x_star_theoretical)
print("Function value at theoretical minimum:", f_star_theoretical)

... Variant: 2
a = 150
b = 2
f0 = 100
n = 3
Theoretical minimum point: [1. 1. 1.]
Function value at theoretical minimum: 100.0
```

Рисунок 2 — Задание параметров варианта и функции Розенброка

3.3. Определение градиента функции Розенброка

После определения целевой функции был реализован градиент функции Розенброка. Градиент необходим для работы всех рассматриваемых методов, так как направление поиска в них определяется на основе первых производных функции.

Для проверки корректности вычисления градиента он был рассчитан в теоретической точке минимума $x^* = (1, 1, 1)$. В этой точке все компоненты градиента равны нулю, а норма градиента также равна 0. Это подтверждает, что точка x^* является стационарной точкой функции.

```
▶ # Определение градиента функции Розенброка и проверка в точке минимума

def rosenbrock_gradient(x):
    x = np.array(x, dtype=float)
    grad = np.zeros_like(x)

    for i in range(n - 1):
        grad[i] += -4 * a * x[i] * (x[i + 1] - x[i] ** 2) + 2 * b * (x[i] - 1)
        grad[i + 1] += 2 * a * (x[i + 1] - x[i] ** 2)

    return grad

gradient_at_minimum = rosenbrock_gradient(x_star_theoretical)
gradient_norm_at_minimum = np.linalg.norm(gradient_at_minimum)

print("Gradient at theoretical minimum:", gradient_at_minimum)
print("Gradient norm at theoretical minimum:", gradient_norm_at_minimum)

... Gradient at theoretical minimum: [0. 0. 0.]
    Gradient norm at theoretical minimum: 0.0
```

Рисунок 3 — Проверка градиента функции Розенброка

3.4. Задание начальной точки и параметров алгоритмов

Перед запуском алгоритмов были заданы общие параметры вычислений. В качестве начальной точки была выбрана точка $x_0 = (-1.2, 1.0, 0.5)$. Также были заданы критерии остановки: точность по норме градиента $\epsilon = 10^{-6}$, точность по изменению аргумента

$\epsilon_x = 10^{-8}$, точность по изменению значения функции $\epsilon_f = 10^{-10}$ и максимальное количество итераций $\text{max_iter} = 1000$.

Для выбора шага вдоль направления поиска использовалась функция одномерной минимизации. Для методов сопряженных градиентов применялся ограниченный интервал поиска шага, что позволило обеспечить устойчивость вычислений.

```
# Задание общих параметров алгоритмов и функции одномерного поиска

x0 = np.array([-1.2, 1.0, 0.5])

epsilon = 1e-6
epsilon_x = 1e-8
epsilon_f = 1e-10
max_iter = 1000

def line_search(f, x, d):
    result = minimize_scalar(
        lambda alpha: f(x + alpha * d),
        bounds=(0, 0.01),
        method='bounded',
        options={'xatol': 1e-10}
    )
    return result.x

print("Initial point:", x0)
print("Function value at initial point:", rosenbrock_function(x0))
print("Gradient norm at initial point:", np.linalg.norm(rosenbrock_gradient(x0)))
print("epsilon =", epsilon)
print("epsilon_x =", epsilon_x)
print("epsilon_f =", epsilon_f)
print("max_iter =", max_iter)
```

```
... Initial point: [-1.2  1.   0.5]
Function value at initial point: 176.22
Gradient norm at initial point: 395.9032204971311
epsilon = 1e-06
epsilon_x = 1e-08
epsilon_f = 1e-10
max_iter = 1000
```

Рисунок 4 — Задание начальной точки и параметров алгоритмов

3.5. Реализация метода Флетчера–Ривза

На данном этапе был реализован метод сопряженных градиентов Флетчера–Ривза. В начале работы алгоритма вычисляется градиент функции в начальной точке, после чего направление поиска задается как антиградиент. На каждой итерации определяется шаг одномерным поиском, вычисляется новая точка, новое значение градиента и коэффициент β по формуле Флетчера–Ривза.

Для повышения устойчивости вычислений был использован перезапуск направления. Если направление перестает быть направлением спуска, оно заменяется на антиградиент. Также перезапуск выполняется через каждые n итераций. В результате метод нашел точку, близкую к теоретическому минимуму: $x \approx (1, 1, 1)$, а значение функции составило $f(x) \approx 100$.

```
# Реализация метода сопряженных градиентов Флетчера–Ривза с перезапуском направления
def fletcher_reeves(f, grad_f, x0, epsilon=1e-6, max_iter=1000):
    start_time = time.time()

    x = np.array(x0, dtype=float)
    grad = grad_f(x)
    d = -grad

    history = {
        "x": [x.copy()],
        "f": [f(x)],
        "grad_norm": [np.linalg.norm(grad)]
    }

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < epsilon:
            break

        if np.dot(grad, d) >= 0:
            d = -grad

        alpha = line_search(f, x, d)
        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        beta = np.dot(grad_new, grad_new) / max(np.dot(grad, grad), 1e-20)

        if (k + 1) % n == 0:
            beta = 0

        d = -grad_new + beta * d

        history["x"].append(x_new.copy())
        history["f"].append(f(x_new))
        history["grad_norm"].append(np.linalg.norm(grad_new))

        if np.linalg.norm(x_new - x) < epsilon_x and abs(f(x_new) - f(x)) < epsilon_f:
            x = x_new
            grad = grad_new
            break

    x = x_new
    grad = grad_new

    execution_time = time.time() - start_time

    return {
        "method": "Fletcher-Reeves",
        "x_min": x,
        "f_min": f(x),
        "iterations": len(history["f"]) - 1,
        "grad_norm": np.linalg.norm(grad_f(x)),
        "time": execution_time,
        "history": history
    }

fr_result = fletcher_reeves(rosenbrock_function, rosenbrock_gradient, x0, epsilon, max_iter)

print("Method:", fr_result["method"])
print("Found minimum point:", fr_result["x_min"])
print("Function value:", fr_result["f_min"])
print("Iterations:", fr_result["iterations"])
print("Gradient norm:", fr_result["grad_norm"])
print("Execution time:", fr_result["time"])
```

Рисунок 5 — Код реализации метода Флетчера–Ривза


```
... Method: Fletcher-Reeves  
Found minimum point: [0.99999852 0.99999703 0.99999405]  
Function value: 100.00000000002201  
Iterations: 661  
Gradient norm: 8.377764176818737e-06  
Execution time: 0.3449580669403076
```

Рисунок 6 — Результат работы метода Флетчера–Ривза

3.6. Реализация метода Полака–Рибьера

На следующем этапе был реализован метод сопряженных градиентов Полака–Рибьера. Данный метод является модификацией метода Флетчера–Ривза и отличается способом вычисления коэффициента β . В методе Полака–Рибьера коэффициент определяется с учетом разности градиентов на текущей и предыдущей итерациях.

В реализации также использовался перезапуск направления поиска. Если полученное направление переставало быть направлением спуска, оно заменялось на направление антиградиента. Кроме того, отрицательные значения коэффициента β заменялись на 0, что повышает устойчивость метода при минимизации неквадратичных функций.

В результате метод Полака–Рибьера нашел точку, близкую к теоретическому минимуму $x \approx (1, 1, 1)$. Значение функции в найденной точке составило $f(x) \approx 100$, при этом количество итераций оказалось меньше, чем у метода Флетчера–Ривза.


```

# Реализация метода сопряженных градиентов Полака-Рибьера с перезапуском направления

def polak_ribiere(f, grad_f, x0, epsilon=1e-6, max_iter=1000):
    start_time = time.time()

    x = np.array(x0, dtype=float)
    grad = grad_f(x)
    d = -grad

    history = {
        "x": [x.copy()],
        "f": [f(x)],
        "grad_norm": [np.linalg.norm(grad)]
    }

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < epsilon:
            break

        if np.dot(grad, d) >= 0:
            d = -grad

        alpha = line_search(f, x, d)
        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        y = grad_new - grad
        beta = np.dot(grad_new, y) / max(np.dot(grad, grad), 1e-20)
        beta = max(beta, 0)

        if (k + 1) % n == 0:
            beta = 0

        d = -grad_new + beta * d

        history["x"].append(x_new.copy())
        history["f"].append(f(x_new))
        history["grad_norm"].append(np.linalg.norm(grad_new))

        if np.linalg.norm(x_new - x) < epsilon_x and abs(f(x_new) - f(x)) < epsilon_f:
            x = x_new
            grad = grad_new
            break

    x = x_new
    grad = grad_new

    execution_time = time.time() - start_time

    return {
        "method": "Polak-Ribiere",
        "x_min": x,
        "f_min": f(x),
        "iterations": len(history["f"]) - 1,
        "grad_norm": np.linalg.norm(grad_f(x)),
        "time": execution_time,
        "history": history
    }

pr_result = polak_ribiere(rosenbrock_function, rosenbrock_gradient, x0, epsilon, max_iter)

print("Method:", pr_result["method"])
print("Found minimum point:", pr_result["x_min"])
print("Function value:", pr_result["f_min"])
print("Iterations:", pr_result["iterations"])
print("Gradient norm:", pr_result["grad_norm"])
print("Execution time:", pr_result["time"])

```

Рисунок 7 — Код реализации метода Полака–Рибьера

```

... Method: Polak-Ribiere
Found minimum point: [0.99999911 0.99999822 0.99999642]
Function value: 100.000000000000796
Iterations: 314
Gradient norm: 3.908735100114691e-06
Execution time: 0.1497941017150879

```

Рисунок 8 — Результат работы метода Полака–Рибьера

3.7. Реализация метода Девидона–Флетчера–Пауэлла

Далее был реализован квазиньютоновский метод Девидона–Флетчера–Пауэлла (DFP). В отличие от методов сопряженных градиентов, данный метод использует аппроксимацию обратной матрицы Гессе. На первой итерации эта матрица задается как единичная, после чего на каждой итерации обновляется с использованием векторов изменения аргумента и изменения градиента.

Направление поиска в методе DFP определяется как произведение аппроксимации обратной матрицы Гессе на антиградиент. Для выбора шага использовался одномерный поиск для квазиньютоновских методов. В случае нарушения условий положительной определенности матрица аппроксимации сбрасывалась к единичной матрице для сохранения устойчивости вычислений.

В результате метод DFP быстро сошелся к точке, близкой к теоретическому минимуму $x \approx (1, 1, 1)$. Значение функции в найденной точке составило $f(x) = 100$, а количество итераций было значительно меньше, чем у методов сопряженных градиентов.

```

# Реализация квазиньютоновского метода Девидона-Флетчера-Пауэлла

def dfp_method(f, grad_f, x0, epsilon=1e-6, max_iter=1000):
    start_time = time.time()

    x = np.array(x0, dtype=float)
    H = np.eye(len(x))
    grad = grad_f(x)

    history = {
        "x": [x.copy()],
        "f": [f(x)],
        "grad_norm": [np.linalg.norm(grad)]
    }

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < epsilon:
            break

        d = -H @ grad

        if np.dot(grad, d) >= 0:
            H = np.eye(len(x))
            d = -grad

        alpha = line_search_quasi_newton(f, x, d)
        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        s = x_new - x
        y = grad_new - grad

        sy = np.dot(s, y)
        Hy = H @ y
        yHy = np.dot(y, Hy)

        if sy > 1e-12 and yHy > 1e-12:
            H = H + np.outer(s, s) / sy - np.outer(Hy, Hy) / yHy
        else:
            H = np.eye(len(x))

        history["x"].append(x_new.copy())
        history["f"].append(f(x_new))
        history["grad_norm"].append(np.linalg.norm(grad_new))

        if np.linalg.norm(x_new - x) < epsilon_x and abs(f(x_new) - f(x)) < epsilon_f:
            x = x_new
            grad = grad_new
            break

    x = x_new
    grad = grad_new

    execution_time = time.time() - start_time

    return {
        "method": "DFP",
        "x_min": x,
        "f_min": f(x),
        "iterations": len(history["f"]) - 1,
        "grad_norm": np.linalg.norm(grad_f(x)),
        "time": execution_time,
        "history": history
    }

dfp_result = dfp_method(rosenbrock_function, rosenbrock_gradient, x0, epsilon, max_iter)

print("Method:", dfp_result["method"])
print("Found minimum point:", dfp_result["x_min"])
print("Function value:", dfp_result["f_min"])
print("Iterations:", dfp_result["iterations"])
print("Gradient norm:", dfp_result["grad_norm"])
print("Execution time:", dfp_result["time"])

```

Рисунок 9 — Код реализации метода DFP

```

... Method: DFP
Found minimum point: [1.          1.00000001 1.00000003]
Function value: 100.0
Iterations: 44
Gradient norm: 2.4293243339881603e-06
Execution time: 0.0348515510559082

```

Рисунок 10 — Результат работы метода DFP

3.8. Реализация метода BFGS

На следующем этапе был реализован квазиньютоновский метод BFGS. Как и метод DFP, он использует аппроксимацию обратной матрицы Гессе, однако формула обновления матрицы отличается и на практике обычно обладает более высокой численной устойчивостью.

На каждой итерации алгоритм вычисляет направление поиска, выполняет одномерную минимизацию для нахождения оптимального шага, после чего обновляет текущую точку и градиент. Затем на основе векторов s и y выполняется обновление матрицы H по формуле BFGS. Если направление поиска перестает быть направлением спуска, матрица H заменяется на единичную, а направление выбирается как антиградиент.

В результате метод BFGS нашел точку $x = (1, 1, 1)$, значение функции составило $f(x) = 100$. Метод показал наименьшее количество итераций среди реализованных алгоритмов и высокую точность по норме градиента.

```

# Реализация квазиньютоновского метода BFGS

def bfgs_method(f, grad_f, x0, epsilon=1e-6, max_iter=1000):
    start_time = time.time()

    x = np.array(x0, dtype=float)
    H = np.eye(len(x))
    grad = grad_f(x)

    history = {
        "x": [x.copy()],
        "f": [f(x)],
        "grad_norm": [np.linalg.norm(grad)]
    }

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < epsilon:
            break

        d = -H @ grad

        if np.dot(grad, d) >= 0:
            H = np.eye(len(x))
            d = -grad

        alpha = line_search_quasi_newton(f, x, d)
        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        s = x_new - x
        y = grad_new - grad
        sy = np.dot(s, y)

        if sy > 1e-12:
            rho = 1.0 / sy
            I = np.eye(len(x))
            H = (I - rho * np.outer(s, y)) @ H @ (I - rho * np.outer(y, s)) + rho * np.outer(s, s)
        else:
            H = np.eye(len(x))

        history["x"].append(x_new.copy())
        history["f"].append(f(x_new))
        history["grad_norm"].append(np.linalg.norm(grad_new))

        if np.linalg.norm(x_new - x) < epsilon_x and abs(f(x_new) - f(x)) < epsilon_f:
            x = x_new
            grad = grad_new
            break

        x = x_new
        grad = grad_new

    execution_time = time.time() - start_time

    return {
        "method": "BFGS",
        "x_min": x,
        "f_min": f(x),
        "iterations": len(history["f"]) - 1,
        "grad_norm": np.linalg.norm(grad_f(x)),
        "time": execution_time,
        "history": history
    }

bfgs_result = bfgs_method(rosenbrock_function, rosenbrock_gradient, x0, epsilon, max_iter)

print("Method:", bfgs_result["method"])
print("Found minimum point:", bfgs_result["x_min"])
print("Function value:", bfgs_result["f_min"])
print("Iterations:", bfgs_result["iterations"])
print("Gradient norm:", bfgs_result["grad_norm"])
print("Execution time:", bfgs_result["time"])

```

Рисунок 11 — Код реализации метода BFGS

```

... Method: BFGS
Found minimum point: [1. 1. 1.]
Function value: 100.0
Iterations: 38
Gradient norm: 2.6691543704683736e-10
Execution time: 0.02639007568359375

```

Рисунок 12 — Результат работы метода BFGS

3.9. Реализация метода L-BFGS

Далее был реализован метод L-BFGS, который является модификацией метода BFGS с ограниченной памятью. В отличие от обычного BFGS, данный метод не хранит полную матрицу аппроксимации обратной матрицы Гессе. Вместо этого сохраняются только несколько последних пар векторов s и y , что позволяет уменьшить требования к памяти.

Для вычисления направления поиска использовалась двухпетлевая рекурсия. На каждой итерации выполнялся одномерный поиск шага, после чего обновлялась текущая точка, вычислялся новый градиент и сохранялась новая пара векторов. В данной работе размер памяти был выбран равным $m = 5$.

В результате метод L-BFGS также сошелся к точке, близкой к теоретическому минимуму $x \approx (1, 1, 1)$. Значение функции составило $f(x) = 100$. По количеству итераций и времени выполнения метод оказался близок к BFGS и DFP.

```
# Реализация квазиньютоновского метода L-BFGS с ограниченной памятью

def lbfgs_method(f, grad_f, x0, epsilon=1e-6, max_iter=1000, m=5):
    start_time = time.time()

    x = np.array(x0, dtype=float)
    grad = grad_f(x)

    s_list = []
    y_list = []

    history = {
        "x": [x.copy()],
        "f": [f(x)],
        "grad_norm": [np.linalg.norm(grad)]
    }

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < epsilon:
            break

        q = grad.copy()
        alpha_list = []
        rho_list = []

        for i in range(len(s_list) - 1, -1, -1):
            s = s_list[i]
            y = y_list[i]
            rho = 1.0 / max(np.dot(y, s), 1e-20)
            alpha = rho * np.dot(s, q)
            q = q - alpha * y
            alpha_list.append(alpha)
            rho_list.append(rho)

        if len(s_list) > 0:
            s_last = s_list[-1]
            y_last = y_list[-1]
            gamma = np.dot(s_last, y_last) / max(np.dot(y_last, y_last), 1e-20)
            r = gamma * q
        else:
            r = q

        alpha_list = alpha_list[::-1]
        rho_list = rho_list[::-1]

        for i in range(len(s_list)):
```

```

for i in range(len(s_list)):
    s = s_list[i]
    y = y_list[i]
    rho = rho_list[i]
    alpha = alpha_list[i]
    beta = rho * np.dot(y, r)
    r = r + s * (alpha - beta)

d = -r

if np.dot(grad, d) >= 0:
    d = -grad
    s_list = []
    y_list = []

alpha_step = line_search_quasi_newton(f, x, d)
x_new = x + alpha_step * d
grad_new = grad_f(x_new)

s_new = x_new - x
y_new = grad_new - grad

if np.dot(s_new, y_new) > 1e-12:
    s_list.append(s_new)
    y_list.append(y_new)

    if len(s_list) > m:
        s_list.pop(0)
        y_list.pop(0)

history["x"].append(x_new.copy())
history["f"].append(f(x_new))
history["grad_norm"].append(np.linalg.norm(grad_new))

if np.linalg.norm(x_new - x) < epsilon_x and abs(f(x_new) - f(x)) < epsilon_f:
    x = x_new
    grad = grad_new
    break

x = x_new
grad = grad_new

execution_time = time.time() - start_time

return {
    "method": "L-BFGS",
    "x_min": x,
    "f_min": f(x),
    "iterations": len(history["f"]) - 1,
    "grad_norm": np.linalg.norm(grad_f(x)),
    "time": execution_time,
    "history": history
}

lbfgs_result = lbfgs_method(rosenbrock_function, rosenbrock_gradient, x0, epsilon, max_iter, m=5)

print("Method:", lbfgs_result["method"])
print("Found minimum point:", lbfgs_result["x_min"])
print("Function value:", lbfgs_result["f_min"])
print("Iterations:", lbfgs_result["iterations"])
print("Gradient norm:", lbfgs_result["grad_norm"])
print("Execution time:", lbfgs_result["time"])

```

Рисунок 13 — Код реализации метода L-BFGS

```

... Method: L-BFGS
Found minimum point: [0.99999999 0.99999998 0.99999995]
Function value: 100.0
Iterations: 43
Gradient norm: 8.67507814786309e-07
Execution time: 0.029248952865600586

```

Рисунок 14 — Результат работы метода L-BFGS

3.10. Сравнение результатов работы алгоритмов

После реализации всех методов была сформирована сравнительная таблица результатов. В таблицу были включены найденная точка минимума, значение функции в этой точке, количество итераций, норма градиента и время выполнения каждого алгоритма.

Из полученных результатов видно, что все реализованные методы нашли точку, близкую к теоретическому минимуму $x^* = (1, 1, 1)$, а значение функции во всех случаях оказалось близким к $f(x^*) = 100$. Наибольшее количество итераций потребовалось методу Флетчера–Ривза. Метод Полака–Рибьера показал более быструю сходимость по сравнению с методом Флетчера–Ривза.

Квазиньютоновские методы DFP, BFGS и L-BFGS показали более высокую эффективность. Среди них метод BFGS продемонстрировал наименьшее количество итераций и наилучшую точность по норме градиента.

▶ # Формирование сравнительной таблицы результатов работы алгоритмов

```
results = [fr_result, pr_result, dfp_result, bfgs_result, lbfgs_result]

comparison_table = pd.DataFrame({
    "Метод": [result["method"] for result in results],
    "x*": [np.round(result["x_min"], 8) for result in results],
    "f(x*)": [result["f_min"] for result in results],
    "Количество итераций": [result["iterations"] for result in results],
    "Норма градиента": [result["grad_norm"] for result in results],
    "Время выполнения, сек": [result["time"] for result in results]
})

comparison_table
```

...	Метод	x*	f(x*)	Количество итераций	Норма градиента	Время выполнения, сек
0	Fletcher-Reeves	[0.99999852, 0.99999703, 0.99999405]	100.0	661	8.377764e-06	0.344958
1	Polak-Ribiere	[0.99999911, 0.99999822, 0.99999642]	100.0	314	3.908735e-06	0.149794
2	DFP	[1.0, 1.00000001, 1.00000003]	100.0	44	2.429324e-06	0.034852
3	BFGS	[1.0, 1.0, 1.0]	100.0	38	2.669154e-10	0.026390
4	L-BFGS	[0.99999999, 0.99999998, 0.99999995]	100.0	43	8.675078e-07	0.029249

Рисунок 15 — Сравнительная таблица результатов алгоритмов

3.11. Анализ сходимости алгоритмов по значению функции

Для анализа поведения алгоритмов был построен график изменения значения целевой функции по итерациям. Данный график позволяет оценить, как быстро каждый метод уменьшает значение функции Розенброка в процессе оптимизации.

По графику видно, что все методы постепенно уменьшают значение функции и сходятся к значению, близкому к 100. Метод Флетчера–Ривза требует большего количества итераций, поэтому его кривая сходимости является наиболее длинной. Квазиньютоновские методы достигают минимума значительно быстрее, что подтверждает их более высокую эффективность при решении данной задачи.

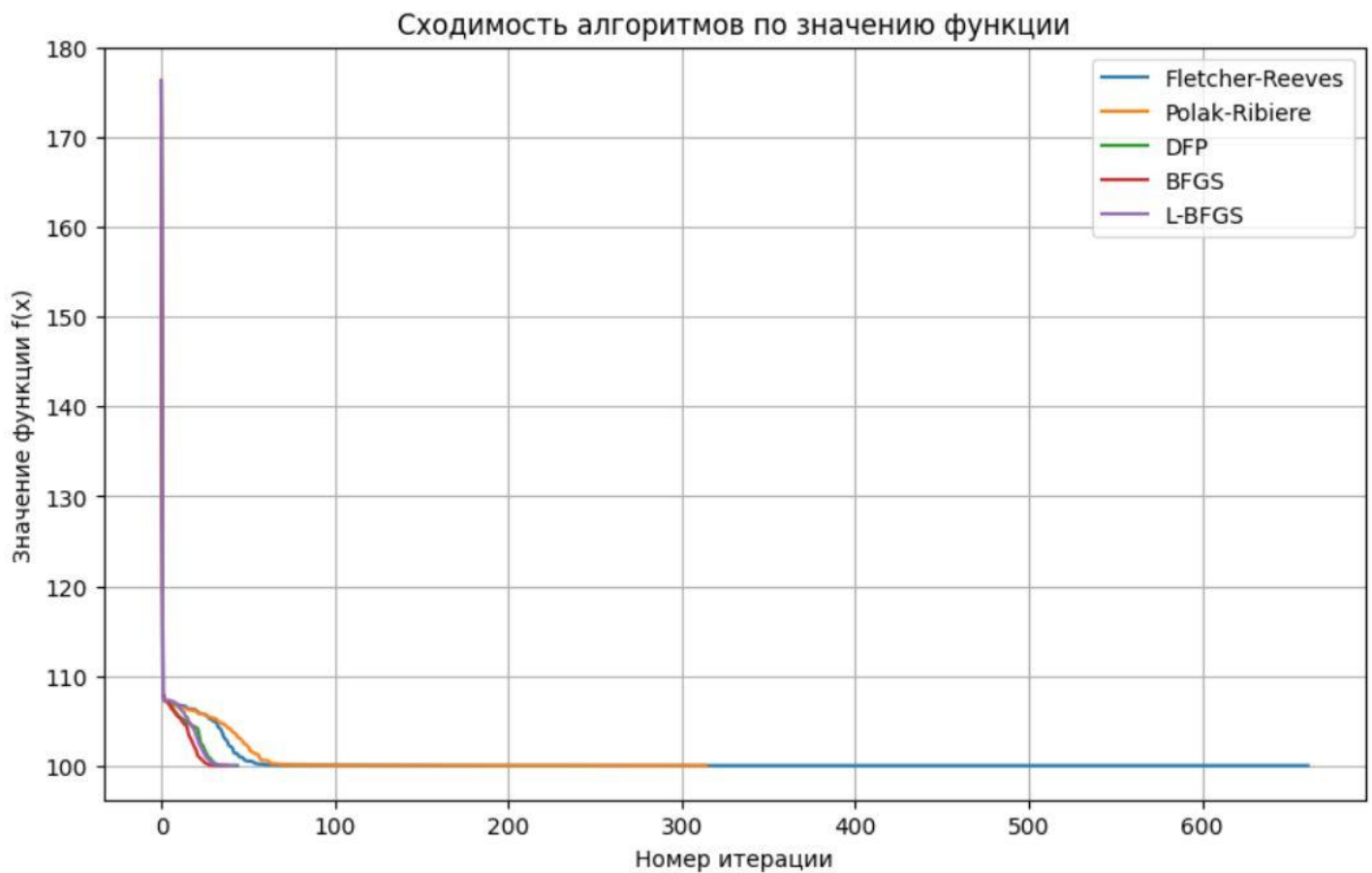


Рисунок 16 — Сходимость алгоритмов по значению функции

3.12. Анализ сходимости алгоритмов по норме градиента

Дополнительно был построен график изменения нормы градиента по итерациям. Норма градиента является важным критерием остановки алгоритмов оптимизации, так как в точке стационарного минимума градиент должен быть близок к нулю.

Для более наглядного сравнения использовалась логарифмическая шкала по оси значения нормы градиента. Из графика видно, что квазиньютоновские методы DFP, BFGS и L-BFGS достаточно быстро уменьшают норму градиента. Наиболее точный результат по норме градиента показал метод BFGS. Методы сопряженных градиентов также сошлись к минимуму, однако потребовали большего количества итераций.

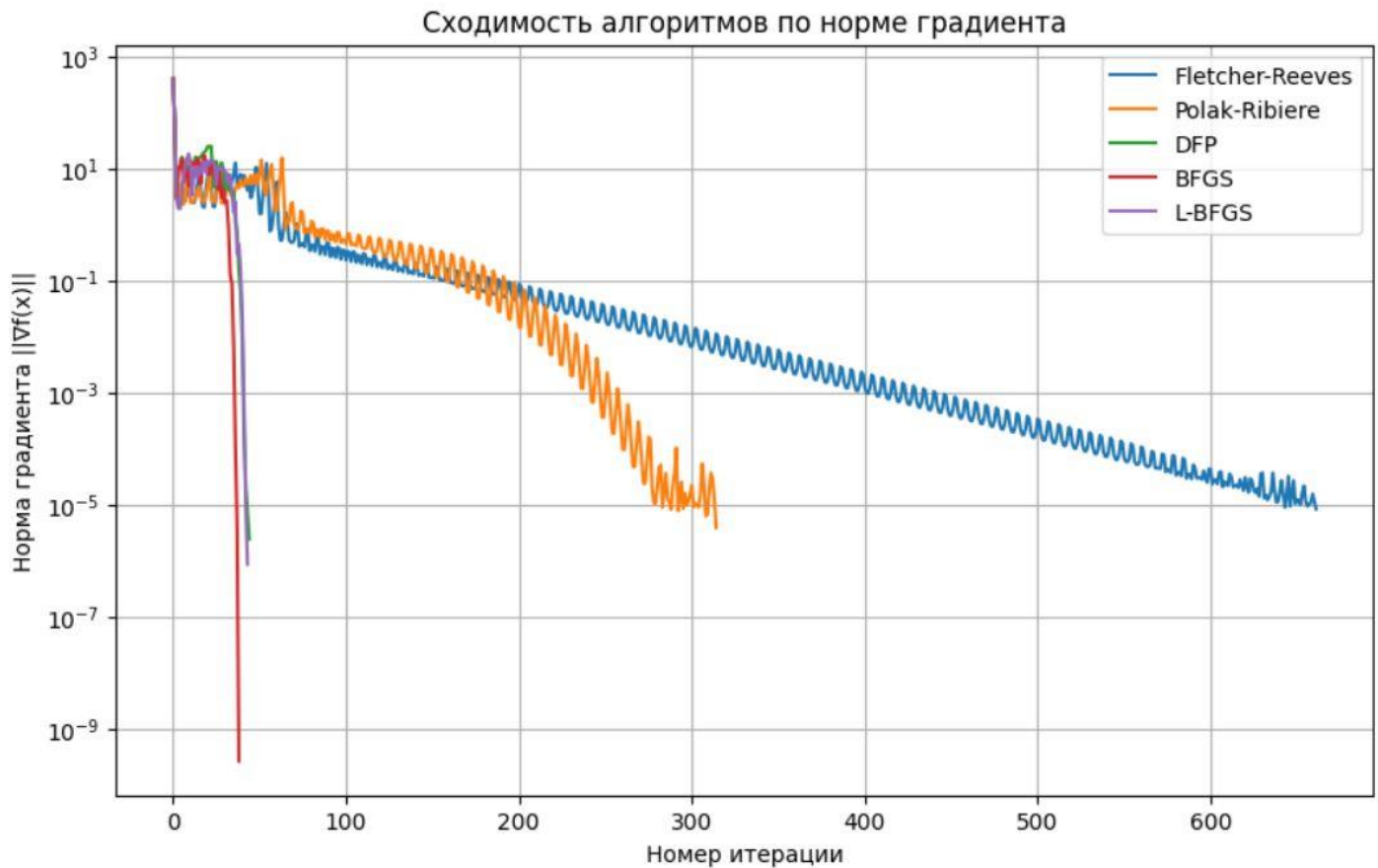


Рисунок 17 — Сходимость алгоритмов по норме градиента

3.13. Сравнение алгоритмов по количеству итераций и времени выполнения

Для итогового сравнения были построены диаграммы, показывающие количество итераций и время выполнения каждого алгоритма. Эти характеристики позволяют оценить практическую эффективность реализованных методов.

По количеству итераций наилучший результат показал метод BFGS, которому потребовалось 38 итераций. Близкие результаты показали методы L-BFGS и DFP. Метод Полака–Рибьера оказался быстрее метода Флетчера–Ривза среди методов сопряженных градиентов. Метод Флетчера–Ривза потребовал наибольшего количества итераций.

По времени выполнения все методы отработали достаточно быстро, так как размерность задачи небольшая. Однако квазиньютоновские методы показали более выгодное соотношение между точностью результата и количеством итераций. В целом наиболее эффективным для данной задачи оказался метод BFGS.

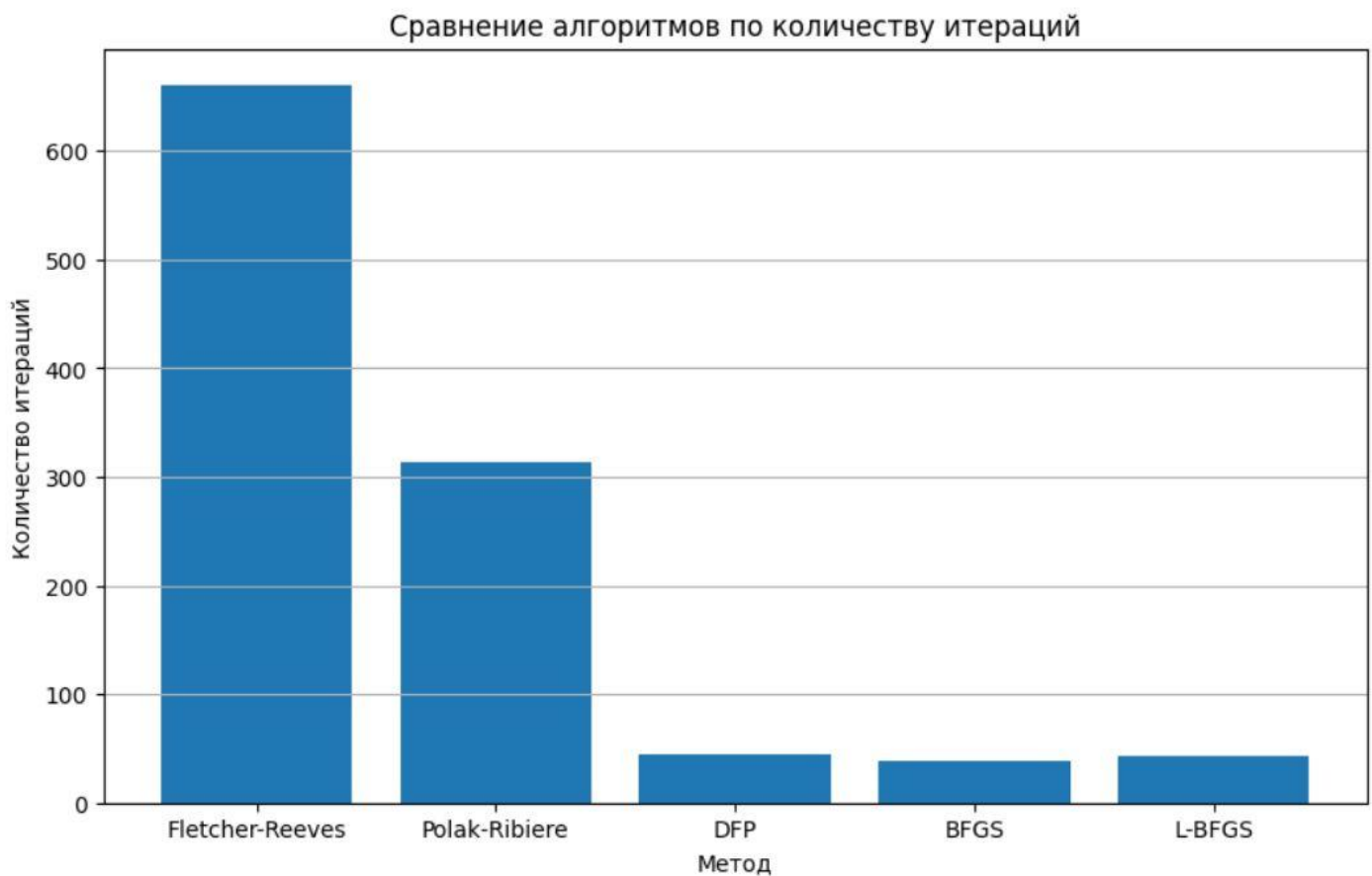


Рисунок 18 — Сравнение алгоритмов по количеству итераций

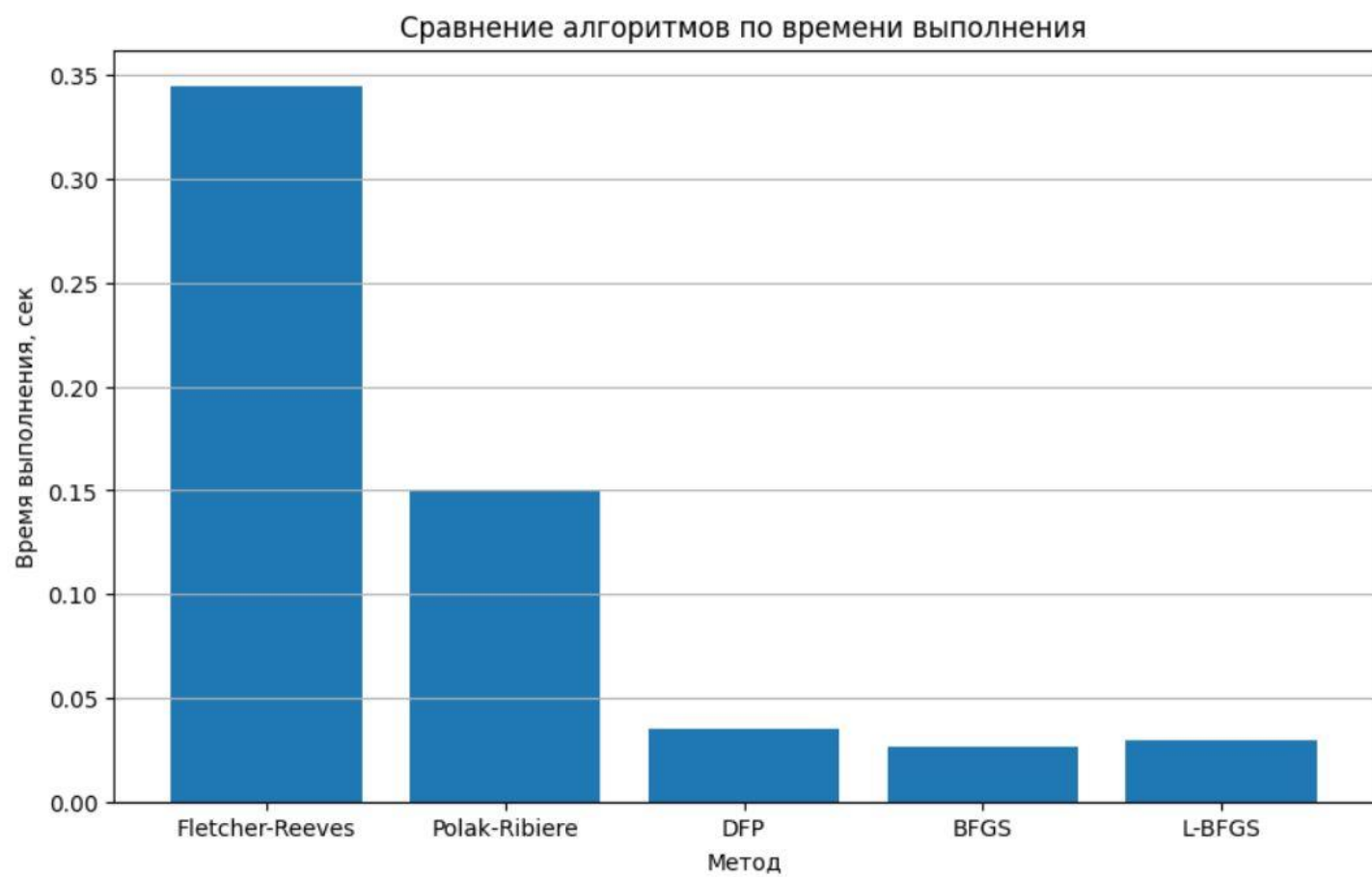


Рисунок 19 — Сравнение алгоритмов по времени выполнения

4) Заключение

В ходе лабораторной работы были изучены и реализованы методы многомерного поиска для минимизации тестовой функции Розенброка. Для варианта 2 были использованы параметры $a = 150$, $b = 2$, $f_0 = 100$, $n = 3$. Теоретическая точка минимума функции имеет вид $x^* = (1, 1, 1)$, а значение функции в этой точке равно $f(x^*) = 100$.

В работе были реализованы два метода сопряженных градиентов: метод Флетчера–Ривза и метод Полака–Рибьера. Также были реализованы квазиньютоновские методы DFP, BFGS и L-BFGS. Для всех алгоритмов использовался одномерный поиск шага вдоль направления спуска и критерии остановки по норме градиента, изменению аргумента и изменению значения функции.

По результатам вычислений все методы нашли точку, близкую к теоретическому минимуму, и значение функции, близкое к 100. Метод Флетчера–Ривза потребовал наибольшего количества итераций. Метод Полака–Рибьера показал более быструю сходимость среди методов сопряженных градиентов.

Квазиньютоновские методы показали более высокую эффективность. Метод DFP сошелся за 44 итерации, метод L-BFGS — за 43 итерации, а метод BFGS показал лучший результат, достигнув минимума за 38 итераций. Кроме того, BFGS обеспечил наименьшую норму градиента, что подтверждает высокую точность найденного решения.

Таким образом, для данной задачи наиболее эффективным оказался метод BFGS. Он продемонстрировал лучшее сочетание точности, количества итераций и времени выполнения.